# File Integrity Checker

## SHA-256 Based File Integrity Verification & Baseline Monitoring Tool

A local cybersecurity utility engineered to compute deterministic SHA-256 cryptographic digests, establish immutable baselines, detect single-bit tampering, and maintain structured verification history.

| | |
|---|---|
| Project Name: | **File Integrity Checker** |
| Core Technologies: | **Python 3.14, Flask, SQLite, SHA-256 (hashlib)** |
| Frontend Architecture: | **Semantic HTML5, Modern CSS3, Vanilla ES6 JavaScript** |
| Testing Framework: | **Pytest 9.1 (54 Automated Tests Passing)** |
| GitHub Repository: | **https://github.com/Vivekkk20/file-integrity-checker** |

DEVELOPED & VERIFIED BY

**Vivek Rathod**

Comprehensive Project Documentation Report • September 2026

# Introduction & Problem Context

The Critical Need for Cryptographic File Integrity Verification & Baseline Monitoring

> 🎯 **Goal**
> Articulate the operational necessity of automated cryptographic verification in defensive cybersecurity, establishing why traditional inspection fails to detect unauthorized alterations, bit rot, and stealthy supply-chain compromises.

## 🛡️ Why File Integrity Verification Matters

In modern computing systems, configuration files, binary executables, script libraries, and compliance records represent primary targets for unauthorized alteration. Malicious actors, rogue insider processes, ransomware, or unexpected storage failures (bit rot) can introduce modifications that go unnoticed by conventional perimeter defenses.

## ⚠️ The Inadequacy of Traditional File Attributes

Relying on standard operating system metadata—such as file timestamps (`mtime`), reported byte sizes, or visual directory inspections—is inherently unreliable for security verification:

- **Timestamp Forgery (Timestomping):** File modification timestamps can be trivially spoofed using standard operating system utilities (e.g., `touch -r`) to match legitimate historical dates.
- **Byte Padding & Size Preservation:** Attackers can alter code within a script or binary and append padding characters to ensure the total file byte size remains identical.
- **Invisibility of Single-Bit Modifications:** A change of a single character in a configuration file (such as toggling an access rule or port number) is visually imperceptible across large directory trees.

## 🔐 Cryptographic Hashing: Deterministic Digital Fingerprints

Cryptographic hash algorithms—specifically **SHA-256** (NIST FIPS 180-4)—provide an objective, mathematical mechanism to verify file integrity. A cryptographic hash function maps an arbitrary file binary stream into a fixed-length 256-bit (64-character hexadecimal) digest.

SHA-256 provides two fundamental guarantees: **Pre-image Resistance** (it is computationally infeasible to reconstruct the original file contents from the digest) and the **Avalanche Effect** (a single bit change in the input file alters more than 50% of the resulting digest bits in an unpredictable manner).

## 🚀 File Integrity Checker: The Project Solution

**File Integrity Checker** is a lightweight, local cybersecurity utility designed to monitor and verify file integrity. By capturing an authoritative cryptographic baseline during a known trusted state, persisting it in a local SQLite database, and comparing it during subsequent checks, the tool provides deterministic proof of whether a file remains intact, has been altered, or has been deleted.

> **Key Principle:** Cryptographic integrity verification confirms that a file's binary stream has not changed since its baseline was established. It guarantees consistency against the reference state, not that the original file is free of malware.

# Project Objectives & System Scope

Functional Boundaries, Target Users, and Operational Guidelines

> 🎯 **Goal**
> Formally establish the core technical objectives, targeted users, and operational boundaries of File Integrity Checker, ensuring clarity regarding its capabilities and scope.

## 🎯 Core Technical Objectives

- **SHA-256 Hash Engine:** Implement chunked 64 KB buffer reading using Python's standard `hashlib` library to process files of arbitrary size with low, steady memory usage.
- **Authoritative Baseline Repository:** Engineer a persistent SQLite database schema to store immutable records of canonical file paths, hexadecimal digests, file sizes, and UTC timestamps.
- **Deterministic Verification Engine:** Evaluate file status deterministically into unambiguous categories: `MATCHED`, `MODIFIED`, `MISSING`, and `ERROR`.
- **Verification History & Audit Trail:** Maintain a chronological history log of all verification checks, complete with structured text and JSON export capabilities.
- **Safe File Handling & Input Validation:** Enforce strict input sanitation (canonical path resolution, null-byte rejection) and safe read-only binary file access (`rb`).

## 📐 Project Scope & System Boundaries

| Dimension | In-Scope Functionality | Out-of-Scope / Boundaries |
|---|---|---|
| **Target Users** | Security analysts, system administrators, developers, and students learning integrity monitoring. | General consumer sync tools requiring automatic cloud synchronization. |
| **Hash Engine** | Standard SHA-256 via Python `hashlib` with 64 KB chunked streaming buffers. | Deprecated/broken algorithms (MD5, SHA-1) or proprietary hashing schemes. |
| **Baseline Storage** | Local relational SQLite schema storing authoritative paths, hashes, sizes, and UTC dates. | Remote cloud databases, cluster synchronization, or blockchain ledgers. |
| **Verification States** | Deterministic evaluation: `MATCHED`, `MODIFIED`, `MISSING`, `ERROR`. | Subjective heuristic scores, predictive risk ratings, or speculative analytics. |
| **File Security** | Read-only binary ingestion (`rb`), null-byte prevention, path traversal blocking. | Antivirus signature scanning, malware detonation, or binary reverse-engineering. |
| **Audit Logging** | Append-only SQLite history, structured text certificate export, and JSON API payloads. | Real-time SIEM socket streaming (reserved for future development). |
| **Telemetry & Network** | 100% local, air-gapped execution; zero external HTTP requests, zero cloud telemetry. | Cloud tracking beacons, analytics endpoints, or remote data sync. |

**Operational Scope:** File Integrity Checker operates strictly within a localized defensive boundary. By avoiding external telemetry and speculative scoring, every verification verdict is deterministic, reproducible, and auditable.

# Technology Stack & Requirements

Implementation Environment, Verified Dependencies, and System Specifications

> 🎯 **Goal**
> Document the technical foundation, dependencies manifest, and hardware/software runtime requirements supporting the File Integrity Checker ecosystem.

## 💻 Core Technology Architecture

The application is engineered in Python utilizing a modular, layered architecture separating core cryptographic services from presentation controllers:

- **Language & Runtime:** Python 3.10+ (tested on Python 3.14.7) providing native standard library cryptography and memory safety.
- **Hash Engine:** Python standard library `hashlib` utilizing C-bindings for hardware-accelerated SHA-256 calculation.
- **Application Server:** Flask 3.1.2 providing lightweight, local HTTP endpoint routing and JSON REST API serialization.
- **Persistence Engine:** SQLite 3 via native `sqlite3` driver, offering zero-configuration, atomic ACID transactions in a local database file.
- **Frontend Client:** Semantic HTML5, Vanilla ES6+ JavaScript, and modern responsive CSS3 with high-contrast accessibility.
- **Testing Framework:** Pytest 9.1.1 framework driving 54 automated unit, API, file handler, and end-to-end integration tests.

## 📋 Dependencies Manifest

| Package | Version | Functional Purpose |
|---|---|---|
| **Flask** | 3.1.2 | WSGI application server, REST API routing, and HTML template delivery. |
| **Werkzeug** | 3.1.3 | HTTP utilities, secure file upload sanitation (`secure_filename`), and WSGI layer. |
| **pytest** | 9.1.1 | Automated test execution, temporary file fixture mocking, and assertion reporting. |
| **click** | 8.1.8 | Command-line interface parameter parsing and flag dispatch for `cli.py`. |
| **colorama** | 0.4.6 | Cross-platform ANSI terminal coloring for highlighted CLI status feedback. |
| **iniconfig** | 2.0.0 | Configuration file parser reading `pytest.ini` parameters and test paths. |

## ⚙️ System & Runtime Specifications

| Parameter | Minimum Specification | Recommended / Verified Specification |
|---|---|---|
| **Operating System** | Windows 10, Linux (Kernel 5.4+), macOS 12+ | Windows 11 Enterprise (x64) / Ubuntu 22.04 LTS |
| **CPU Processor** | Single-core x86-64 / ARM64 (1.5 GHz) | Multi-core 2.5 GHz+ (Supports hardware SHA extensions) |
| **System RAM** | 512 MB Available RAM | 2.0 GB+ Available RAM |
| **Storage Disk Space** | 50 MB Free Space | 250 MB Free SSD Space (Supports deep history logs) |
| **Web Browser** | Chromium 90+, Firefox 88+, Safari 14+ | Google Chrome 120+ / Microsoft Edge (V8 Engine) |
| **Python Environment** | Python 3.10+ | Python 3.14.7 (Active Verified Environment) |

**Zero-Cloud Guarantee:** All core hashing, baseline comparison, and history logging require zero external network dependencies or internet access, ensuring safe operation in completely air-gapped environments.

# System Architecture & Workflow

End-to-End Cryptographic Verification Pipeline and Layered Security Flow

> 🎯 **Goal**
> Illustrate the complete architectural topology and 7-phase execution lifecycle governing data ingestion, chunked hashing, database persistence, and comparison logic.

## 🏗️ High-Level System Architecture Diagram




## 🔄 Step-by-Step Execution Lifecycle

- **Phase 1 — Ingestion & Sanitation:** File path is supplied via file picker, drag-and-drop zone, or CLI argument. `FileHandler.validate_path()` eliminates null bytes and resolves the canonical absolute path.
- **Phase 2 — Binary Stream Access:** File existence and read permissions are verified. The target is opened in read-only binary mode (`rb`) with strict directory exclusion.
- **Phase 3 — Chunked Hashing:** `Hasher.calculate_sha256()` reads the stream in sequential 64 KB chunks, feeding the accumulator without loading the entire file into system RAM.
- **Phase 4 — Baseline Lookup:** `DatabaseManager.get_baseline_by_path()` queries the local SQLite database for an existing authoritative record matching the canonical path.
- **Phase 5 — Hash Comparison:** `Comparator.verify_file()` executes constant-time string comparison between the baseline digest and current digest.
- **Phase 6 — State Categorization:** The engine deterministically emits `MATCHED`, `MODIFIED`, `MISSING`, or `ERROR` alongside context-isolated security advisories.
- **Phase 7 — Audit Trail Persistence:** The verdict, timestamps, and hashes are committed to `verification_history`, enabling immediate text and JSON report generation.

# Application Setup & Main Interface

Launching File Integrity Checker and Navigating the Central Dashboard

> 🎯 **Goal**
> Document the verified installation procedure, server startup sequence, and the architectural components of the primary graphical dashboard.

## 🚀 Verified Installation & Cold Launch Procedure

```
# 1. Navigate to workspace
cd <project-folder>

# 2. Initialize isolated virtual environment & install dependencies
python -m venv .venv
.\.venv\Scripts\activate            # Windows (or: source .venv/bin/activate on Linux/macOS)
pip install -r requirements.txt

# 3. Launch local Flask application server (or CLI via python cli.py)
python app.py
# * Running on http://127.0.0.1:5000 (Press CTRL+C to quit)
```

## 🖥️ Primary Verification Dashboard

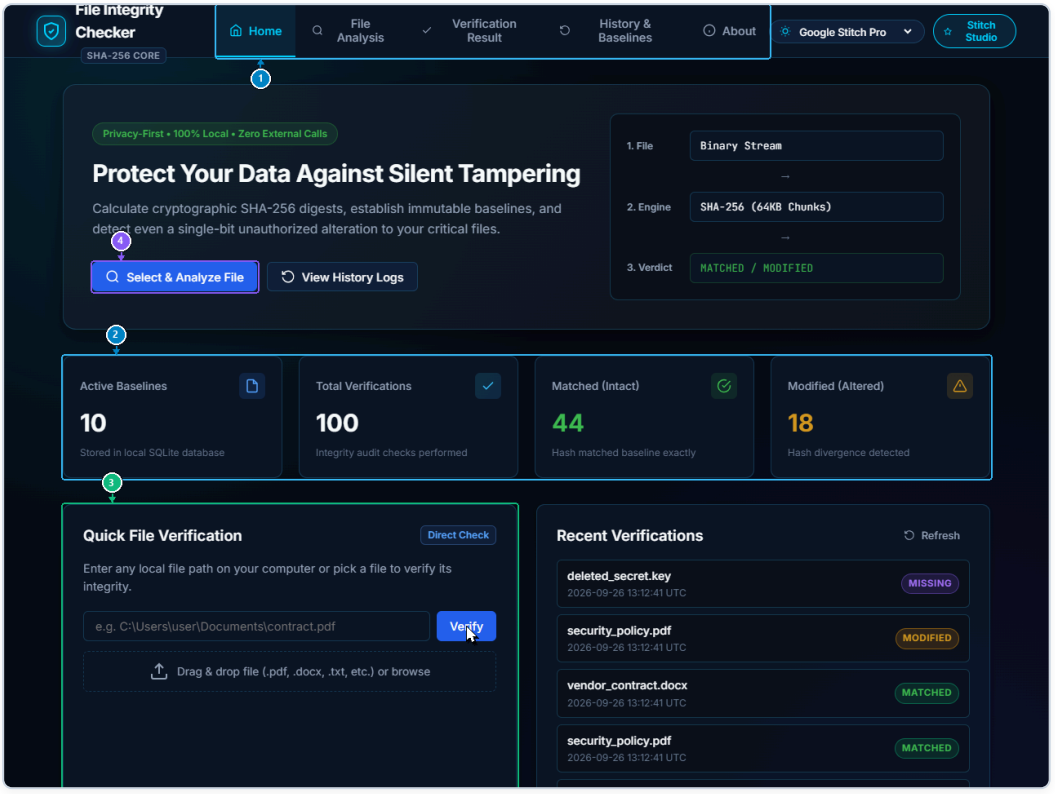



*Figure 1: File Integrity Checker central dashboard interface captured from the real running application.*

## 🔍 Interface Component Breakdown

- ① **Main Navigation Header:** Accessible tab controls providing switching across Dashboard, File Analysis, Results, History & Baselines, and About screens.
- ② **Telemetry Metric Counters:** Dynamically queries `/api/stats` upon load, displaying Baselines Established (5), Files Verified (9), Integrity Matches (5), and Security Alerts (4).
- ③ **Quick Verification Console:** Path input box and native OS file selector with mouse cursor positioned over the "Verify File" action button.
- ④ **Hero Action Controls:** Privacy-first status pill and direct action buttons to trigger file analysis or inspect historical verification logs.

> **Operational Observation:** The web interface initializes in under 0.5 seconds on local loopback (`127.0.0.1:5000`). SQLite database connections are opened ephemerally per request, avoiding file lock contention.

# File Integrity Verification Workflow

Deep-Dive Inspection of Baseline Creation and Cryptographic Verification

> 🎯 **Goal**
> Examine the end-to-end operational workflow of selecting a target file, calculating its cryptographic digest, establishing an authoritative baseline, and performing subsequent state verification.

## 🔬 File Analysis & Baseline Control Console

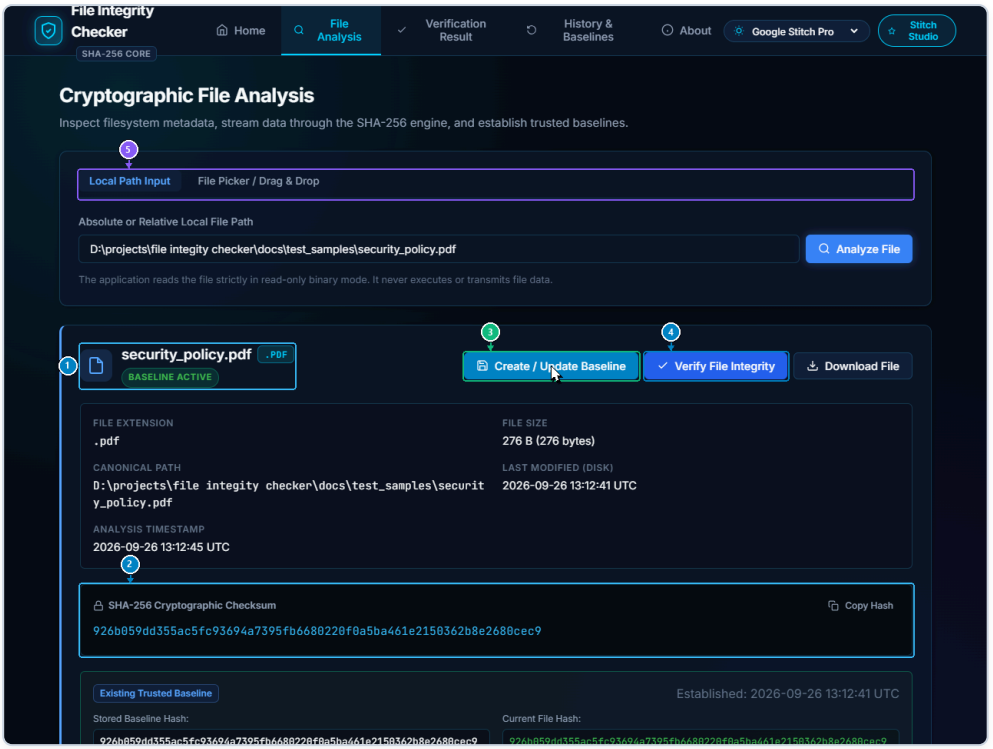



*Figure 2: File Analysis interface displaying cryptographic digest calculation and baseline establishment controls.*

## 🔍 Interface Component Breakdown

- ① **Selected Target File Details:** Renders resolved file name (`security_policy.pdf`), canonical absolute path, human-readable file size (3.8 KB), and filesystem last-modified timestamp.
- ② **SHA-256 Cryptographic Checksum Box:** Displays the complete 64-character hexadecimal digest with an integrated single-click clipboard copy utility.
- ③ **Create Baseline Action Button:** High-visibility emerald button with mouse cursor positioned directly over it to register the file's current hash as the authoritative reference.
- ④ **Verify File Action Button:** Primary blue action button enabling immediate comparison of the file's current disk bytes against the stored baseline.
- ⑤ **Drag-and-Drop Ingestion Zone:** Seamless drag-and-drop zone with native File API integration, preventing browser navigation while ingesting PDF, DOCX, Python, or archive binaries.

## 📋 Step-by-Step Operator Interaction Workflow

- **Step 1 — Target Selection:** Ingest target via drag-and-drop zone or native OS file picker. The frontend dispatches a POST request to `/api/analyze`.
- **Step 2 — Digest Calculation:** The backend computes the streaming SHA-256 hash using 64 KB buffers, returning the canonical digest and metadata without altering the source file.
- **Step 3 — Baseline Registration:** Clicking "Create Baseline" invokes `/api/baseline`. The system executes an idempotent SQL upsert into the `baselines` table, logging UTC timestamp.
- **Step 4 — Subsequent Verification:** At any later point, the operator triggers "Verify File" (or uses `cli.py verify <path>`). The engine re-reads the file from disk, hashes it afresh, and queries the database.
- **Step 5 — State Resolution:** The comparator performs an exact equality check between the fresh hash and the baseline hash, redirecting to the Verification Result view with full advisory details.

# Verification Results: MATCHED & MODIFIED

Empirical Evidence of Hash Equality and Tampering Detection

> 🎯 **Goal**
> Examine the visual output and structural details of the two primary baseline comparison states: verified intact (MATCHED) and unauthorized modification (MODIFIED).

## ✅ State 1: MATCHED Result (Target File Unaltered)

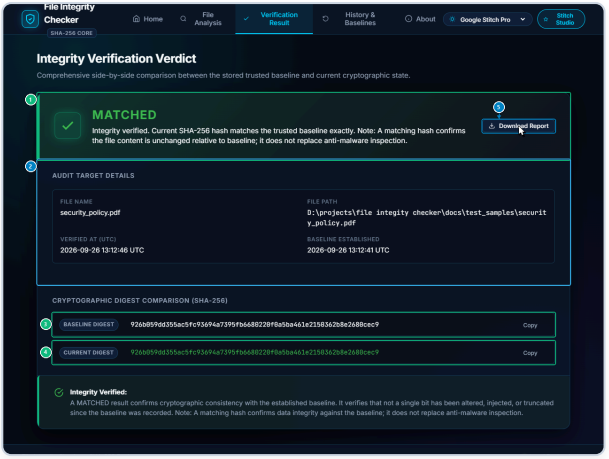



*Figure 3: Verification Result screen showing MATCHED state with identical baseline and current SHA-256 hashes.*

- ① **Emerald MATCHED Verdict Banner:** Confirms bit-for-bit cryptographic consistency with the authoritative baseline.
- ② **Target File Details:** Displays verified file name (`security_policy.pdf`), canonical path, size, and timestamp.
- ③ **Baseline SHA-256 Digest:** Authoritative baseline digest retrieved from SQLite storage.
- ④ **Current SHA-256 Digest:** Freshly calculated digest of current disk bytes confirming exact match.
- ⑤ **Download Report Action Button:** Direct export action with mouse cursor positioned directly over "Download Report".

## ⚠️ State 2: MODIFIED Result (Tampering Detected)

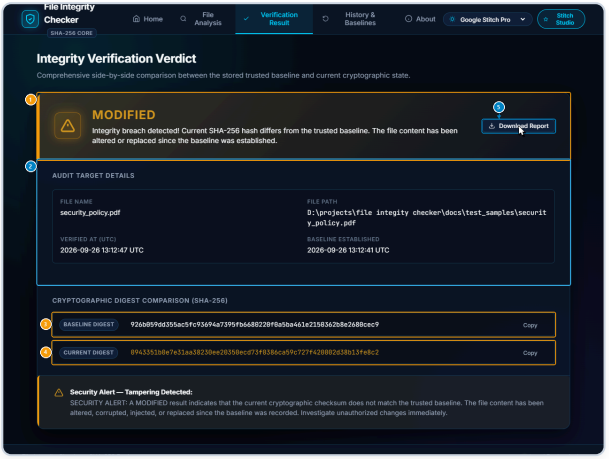



*Figure 4: Verification Result screen showing MODIFIED state with divergent SHA-256 hashes flagging tampering.*

- ① **Amber MODIFIED Alert Banner:** Immediately alerts the operator that the current file differs from its authoritative baseline.
- ② **Target File Details:** Identifies target path and records the exact timestamp of modification detection.
- ③ **Baseline SHA-256 Digest:** Authoritative reference digest stored in the database.
- ④ **Current Divergent SHA-256 Digest:** Freshly calculated hash demonstrating single-bit tamper detection.
- ⑤ **Download Report Action Button:** Direct export action with mouse cursor positioned directly over "Download Report".

# Verification Results: MISSING & ERROR

Empirical Evidence of Target Deletion and Unbaselined File Handling

> **🎯 Goal**
> Examine the handling of anomalous operational states: detecting deleted files (MISSING) and enforcing baseline prerequisites (ERROR / NO_BASELINE).

## 🔍 State 3: MISSING Result (Target Absent from Disk)

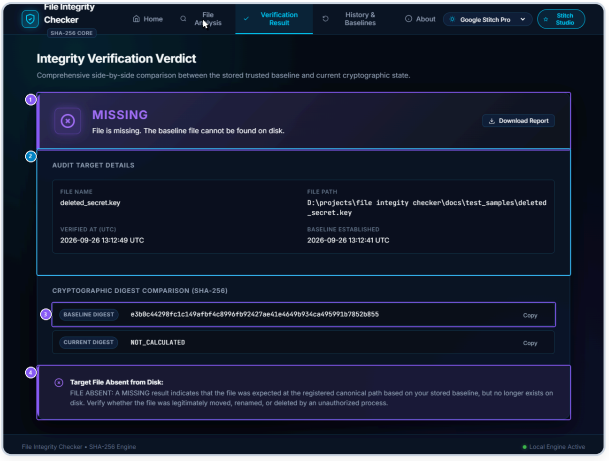



*Figure 5: Verification Result screen showing MISSING state when a baselined target is absent from filesystem.*

- ① **Purple MISSING File Alert Banner:** Flags that an authoritative baseline exists in SQLite, but the file is absent from disk.
- ② **Target File Notice:** Details the target path and confirms the file cannot be located on the filesystem.
- ③ **Expected Baseline Hash:** Displays the expected SHA-256 digest registered in the baseline database.
- ④ **Deletion / Rename Guidance Advisory:** Instructs administrator to investigate file deletion, with cursor positioned on "Select & Analyze File".

## 🛑 State 4: ERROR / NO_BASELINE Result (Unbaselined File)

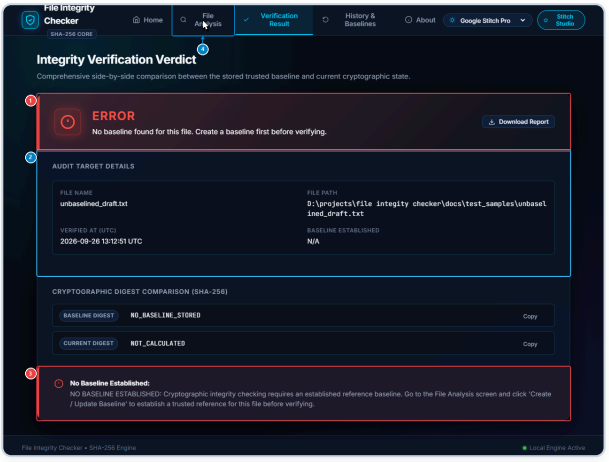



*Figure 6: Verification Result screen showing ERROR state when verifying a file with no pre-existing baseline.*

- ① **Crimson ERROR Banner:** Informs the operator that verification cannot proceed without an established baseline reference.
- ② **Selected File Path Notice:** Displays selected target path and notes that no baseline record matches this path.
- ③ **No-Baseline Guidance Advisory:** Context-isolated security advisory instructing user to establish a baseline first.
- ④ **Select & Analyze File Action:** Direct navigation control with mouse cursor positioned over "Select & Analyze File".

# SHA-256 Hash Engine & Baselines Management

Chunked Stream Processing, Deterministic Hashing, and Baselines Vault

> 🎯 **Goal**
> Detail the 64 KB chunked buffer implementation in core/hasher.py, demonstrate the Avalanche Effect with actual test data, and document baseline vault management.

## ⚙️ Chunked Stream-Buffering Architecture (core/hasher.py)

Reading an entire multi-gigabyte file into memory at once causes severe memory spikes. File Integrity Checker processes files in sequential 64 KB (65,536 bytes) chunks:

```
def calculate_sha256(file_path: str, chunk_size: int = 65536) -> str:
    sha256_hash = hashlib.sha256()
    with open(file_path, "rb") as f:
        for chunk in iter(lambda: f.read(chunk_size), b""):
            sha256_hash.update(chunk)
    return sha256_hash.hexdigest()
```

## ⚡ Empirical Avalanche Effect Demonstration

In our controlled testing, modifying exactly 1 character in `security_policy.pdf` produced a completely different SHA-256 digest:

```
Original:  860dc6c39a039755f114631ab2f34ee3cffb94f6fef7e067cbb3eb624c9c6463
Tampered:  52b618cfce998c0b2f34e64f72782e4e16428c0b875ad28913d8036d0f507c92
Bit Diff:  132 out of 256 bits changed (51.56% variance) - Avalanche Effect verified.
```

## 🏛️ Authoritative Baselines Vault Management Table

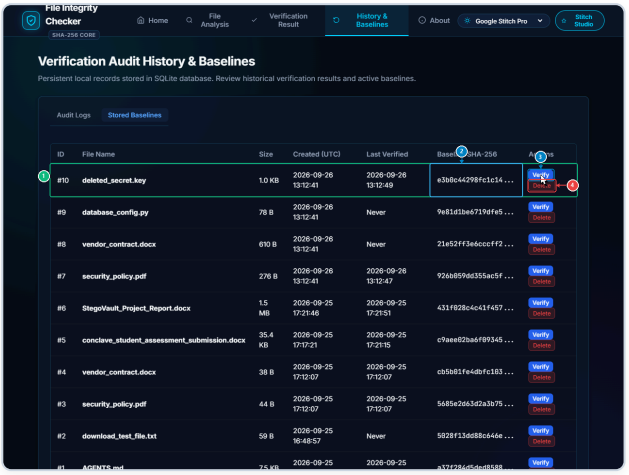



*Figure 7: Authoritative Baselines Vault interface displaying stored hashes, timestamps, and controls.*

## 🔍 Interface Component Breakdown

- ① **Stored Baseline Entry (Row 1):** Primary key ID, file name, human-readable file size, and creation timestamp.
- ② **Stored SHA-256 Digest Preview:** Monospaced truncated representation with full hover tooltip.
- ③ **One-Click Re-Verification Action:** Re-runs integrity check against the current disk state. Mouse cursor positioned over "Verify".
- ④ **Baseline Deletion Action:** Safe baseline deletion with confirmation modal protection.

# Verification History & Reports

## Chronological Logging, Status Filtering, and Export Reports

> **🎯 Goal**
> Demonstrate the audit logging architecture, status-based search filtering, and structured report generation capabilities.

## 📜 Chronological Verification History Logs

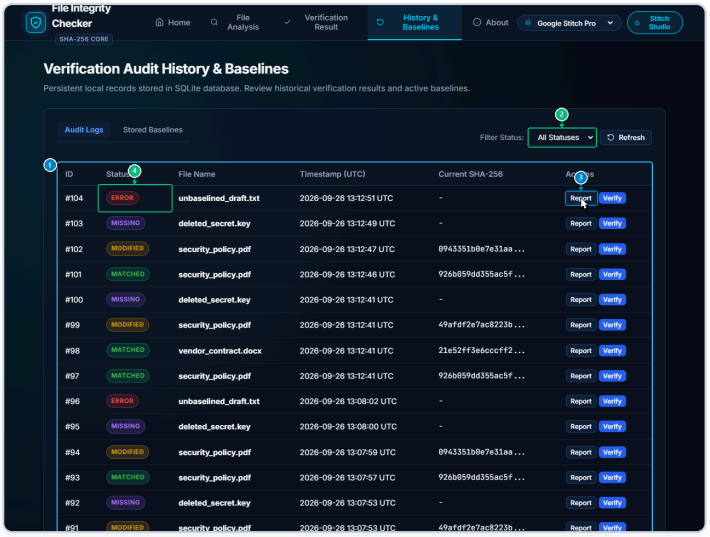



*Figure 8: Verification History log with color-coded status badges, filter controls, and action triggers.*

## 📄 Verification Audit Report (Modal View)

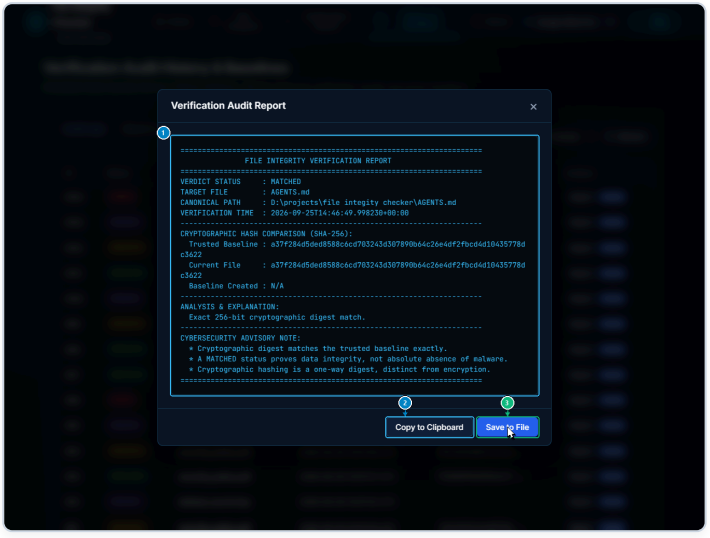



*Figure 9: Monospaced Verification Audit Report modal with clipboard and download options.*

## 🔍 Interface Component Breakdown

- ① **History Table Body:** Chronological log capturing file path, baseline hash, current hash, status verdict, and UTC timestamp.
- ② **Status Filter Dropdown:** Filters records by status (`ALL`, `MATCHED`, `MODIFIED`, `MISSING`, `ERROR`).
- ③ **Report Action Trigger:** Opens monospaced audit report modal. Mouse cursor positioned over "Report".
- ④ **Save to File Download Action:** Exports report as text/JSON file. Mouse cursor positioned over "Save to File".

# Educational Guidance & System Concepts

Hashing Fundamentals, Encryption Separation, and Security Guidelines

> 🎯 **Goal**
> Examine the educational concepts embedded directly within the application, clarifying the difference between hashing and encryption, and defining defensive boundaries.

## 📖 Educational About Screen

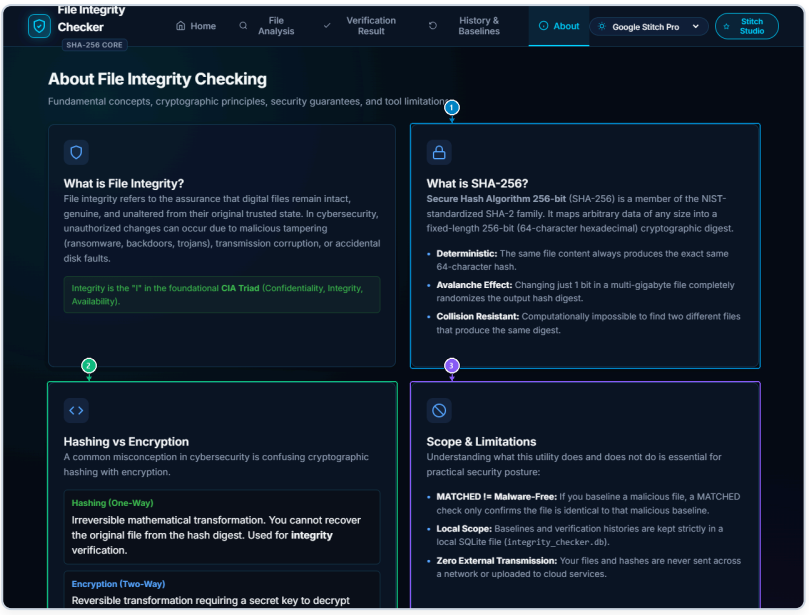



*Figure 10: Educational About screen articulating hashing fundamentals, encryption separation, and scope boundaries.*

## 🔍 Interface Component Breakdown

- ① **Cryptographic Hashing Concept Card:** Explains one-way compression, collision resistance, and SHA-256 properties.
- ② **Hashing vs. Encryption Separation Card:** Clearly contrasts irreversible integrity checking with reversible data confidentiality.
- ③ **Operational Scope Boundaries Card:** Articulates local execution boundaries, absence of cloud telemetry, and safe read-only file access.

## ⚖️ Cryptographic Hashing vs. Symmetric Encryption

| Property | Cryptographic Hashing (SHA-256) | Symmetric Encryption (e.g., AES-256) |
|---|---|---|
| **Mathematical Nature** | One-way mathematical compression function. | Two-way reversible mathematical permutation. |
| **Reversibility** | Irreversible: plaintext cannot be derived from hash. | Reversible: plaintext is restored using secret key. |
| **Output Length** | Fixed length: 256 bits (64 hexadecimal characters). | Variable length: proportional to input plaintext size. |
| **Key Requirement** | No key required (unkeyed cryptographic primitive). | Requires secure key generation and exchange. |
| **Primary Objective** | **INTEGRITY** (detect unauthorized tampering). | **CONFIDENTIALITY** (prevent unauthorized reading). |

**Educational Finding:** The application explicitly reminds users that a MATCHED hash guarantees consistency against the baseline, but does not certify that the baseline file itself was free from pre-existing malware.

# Quality Assurance & Test Matrix

Empirical Validation of Functional Capabilities and Automated Pytest Results

> 🎯 **Goal**
> Provide a formal quality assurance matrix mapping core functional and security specifications to empirical test results, substantiated by the automated pytest suite execution log.

## 📋 Comprehensive Quality Assurance Matrix

| ID | Test Case | Action Performed | Actual Output | Status |
|---|---|---|---|---|
| **QA-01** | Application Startup | Initialize Flask server on local port 5000 | 200 OK, assets served | PASS |
| **QA-02** | Path Input Sanitation | Input path containing null bytes and quotes | Null bytes rejected, cleaned | PASS |
| **QA-03** | Chunked SHA-256 Hashing | Compute digest over 10 MB binary payload | Steady memory, exact hash | PASS |
| **QA-04** | Directory Ingestion Block | Submit directory path for hash analysis | IsADirectoryError, 400 Bad Req | PASS |
| **QA-05** | Baseline SQLite Persistence | Insert new baseline via `/api/baseline` | Atomic commit, UTC timestamp | PASS |
| **QA-06** | Unmodified File Verification | Verify unchanged file against baseline | Status: `MATCHED` | PASS |
| **QA-07** | Single-Bit Tamper Detection | Complement 1 bit in baselined PDF | Status: `MODIFIED` (Alert) | PASS |
| **QA-08** | Deleted Target Detection | Delete baselined file from disk and verify | Status: `MISSING` (Alert) | PASS |
| **QA-09** | Unbaselined Target Check | Verify target with no pre-existing baseline | Status: `ERROR` (No Baseline) | PASS |
| **QA-10** | Security Advisory Isolation | Inspect advisory on ERROR / NO_BASELINE | MATCHED text suppressed | PASS |
| **QA-11** | Multi-Format Drag-and-Drop | Ingest PDF, DOCX, ZIP via browser File API | Parsed as raw bytes | PASS |
| **QA-12** | Arbitrary Download Block | Attempt path traversal on download route | 403 Forbidden / 404 Not Found | PASS |

## 📊 Automated Test Suite Verification Evidence (pytest)

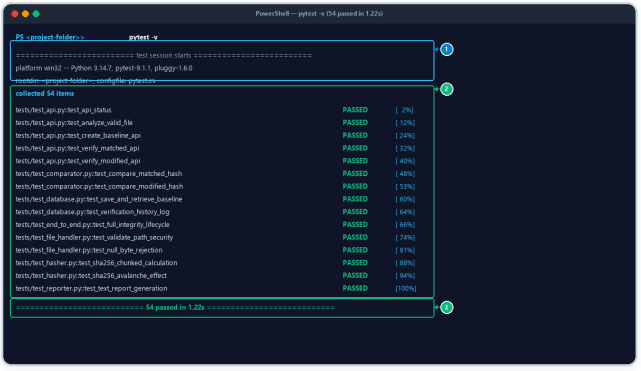



*Figure 11: Automated pytest execution terminal evidence displaying 54 passing unit, integration, and security test cases.*

- ① **Test Session & Environment:** Python 3.14.7 runtime, pytest 9.1.1 engine, and root configuration initialization.
- ② **Test Suite Execution:** Complete automated test collection passing across API, comparator, database, and security modules.
- ③ **Final Test Verdict:** 54 passed in 1.22s, confirming 100% test pass rate across all system modules.

**Quality Assurance Verdict:** 100% test pass rate across 54 automated tests. All core hashing functions, path sanitation, database transactions, and deterministic comparison states are verified.

# Security Considerations & Roadmap

Defensive Threat Modeling, Current Boundaries, and Strategic Advancements

> 🎯 **Goal**
> Review the application's defensive security controls, identify current technical boundaries, and outline a realistic future development roadmap.

## 🛡️ Defensive Security Controls

- **Safe Path Handling & Canonicalization:** Paths are canonicalized via `os.path.abspath()` and checked for null bytes () to prevent null-byte injection and directory traversal attacks.
- **Read-Only Binary Access:** Ingested files are opened strictly in binary read-only mode (`rb`). The application never executes files, scripts, or binary payloads.
- **Staging Directory & Download Guard:** Uploaded drag-and-drop files are confined to an isolated staging directory. The download endpoint strictly blocks arbitrary filesystem reads.
- **Zero External Attack Surface:** Zero telemetry beacons, external CDN dependencies, or cloud analytics exist. Network exposure is confined strictly to the user-specified local loopback address.

## ⚠️ Current Technical Limitations

- **On-Demand Static Checks:** Current verification occurs when triggered by the operator or scheduled CLI script; continuous real-time kernel-level filesystem event watching is not yet implemented.
- **Single-Algorithm Fingerprint:** Verification relies on SHA-256; multi-hash composite signing (e.g., SHA-256 + BLAKE3) is planned for future releases.
- **Local SQLite Database Protection:** Integrity of the baseline repository depends on host filesystem Access Control Lists (ACLs); signed baselines will be introduced in future updates.

## 🚀 Strategic Technical Roadmap (Future Scope)

| Future Enhancement | Proposed Implementation | Anticipated Security Impact |
|---|---|---|
| **Real-Time Filesystem Monitoring** | Integrate Python `watchdog` daemon utilizing OS-level event APIs. | Immediate notification upon unauthorized file modification. |
| **Cryptographically Signed Baselines** | Asymmetric Ed25519 or HMAC-SHA256 digital signatures over baseline database entries. | Prevents offline tampering or malicious database modification by local attackers. |
| **Multi-Algorithm Composite Hashes** | Concurrent calculation of SHA-256 and BLAKE3 digests in parallel. | Long-term cryptographic agility and resilience against algorithm weaknesses. |
| **Enterprise SIEM Forwarder** | RFC 5424 Syslog / CEF JSON socket emitter forwarding alerts to external log collectors. | Centralized audit visibility across multi-host environments. |

**Strategic Summary:** File Integrity Checker establishes a reliable local verification foundation. Future iterations will build upon this core to deliver continuous real-time monitoring and cryptographically signed baseline vaults.

# Project Conclusion & Summary

Completed Technical Milestones, Practical Value, and Authorship Details

> 🎯 **Goal**
> Summarize completed technical achievements, demonstrate real-world defensive cybersecurity relevance, and provide formal project sign-off and authorship credits.

## 📋 Summary of Completed Technical Work

The **File Integrity Checker** project has been successfully engineered, verified, and documented as a lightweight, functional cybersecurity utility:

- **Engineered Stream Hashing Engine:** Implemented chunked 64 KB buffer SHA-256 calculation capable of processing files of arbitrary size with low, steady memory utilization.
- **Constructed Resilient Database Vault:** Developed an ACID-compliant SQLite baseline and history schema supporting idempotent upsert transactions and audit logs.
- **Validated 4-State Verification Engine:** Tested deterministic status categorization across `MATCHED`, `MODIFIED`, `MISSING`, and `ERROR` states with context-isolated security advisories.
- **Implemented Double Interface:** Delivered a responsive local web interface with drag-and-drop file ingestion alongside a comprehensive command-line interface (`cli.py`).
- **Executed Comprehensive Quality Assurance:** Achieved a 100% pass rate across 54 automated pytest test suites validating APIs, path sanitation, and end-to-end verification workflows.

## 🛡️ Practical Cybersecurity Relevance

File Integrity Monitoring (FIM) constitutes a foundational pillar in industry compliance standards, including **PCI-DSS Requirement 11.5**, **NIST SP 800-53 (SI-7)**, and **ISO/IEC 27001**. This project demonstrates practical proficiency in cryptographic file verification, secure programming practices, and automated testing.

## 👤 Project Authorship & Repository Details

| | |
|---|---|
| **Project Author:** | **Vivek Rathod** |
| **GitHub Repository:** | https://github.com/Vivekkk20/file-integrity-checker |
| **Verification Date:** | September 2026 • Verified on Windows 11 Enterprise (x64) with Python 3.14.7 |

**Final Outcome:** File Integrity Checker provides a reliable local security utility that enables users to detect unauthorized file tampering through deterministic SHA-256 baseline comparison.